
ARRAY

Array:

An array is collection of finite, ordered and homogeneous data elements.

Array is finite because it contain limited no of elements. It is ordered as all the elements are stored one by one in contiguous location of computer memory and collection of homogeneous data elements because all the elements of an array are of same data type.

The syntax of an array is:

datatype arrayname[size]

for e.g. : if we want to define an array called **num,** which has 100 elements then the declaration will be

int num[100]

- In above example **‘num’** is an array name with size 100.
- The data type that is used to declare an array is **integer** type. So the array num can store maximum of 100 integer elements.
- The size of an array is also called its length.
- The subscripts of an array are also called index. The index set consist of integers 0,1,2,3,-----,n-1. The
- Length of an array can be evaluated as

Length= upper bound-lower bound+1

For Ex:

An array N[100], the upper bound & lower bound will be

UB=99, LB=0

So,

length=UB-LB+1

= 99-0+1

=100

Array Terminology:

1.Size:

No. of elements in an array is called the size of the array it is also termed as length or dimension.

2. Type:

It means type of an array represents the kind of data type.

For Ex: Array of integers, array of characters

3. Base:

Base of an array is the address of memory location of first element in the array.

4. Index:

all the elements in an array can be referenced by a subscript like Ai or A[i]. This subscript is known index. Index is always an integer value.

5. Range of Index:

Indices of array elements may change from a lower bound(L) to an upper bound(U), which are called the boundaries of an array. In a declaration of an array in ‘C’ language (int A[100]), the range of index is from 0 to 99

One-Dimension Array:

If only one sub-script /index / dimensions is required to reference all the elements in an array then the array will be termed as one-dimensional array.

Representation of linear Array in memory:

Let **LA** be a linear array in the memory of the computer. Let use the following notation:

LOC (LA[K])= address of the kth element of the array**[LA]**

The elements of LA are stored in successive memory cell. Computer does not keep track of the address of every element of **LA,** but needs to keep track only of first element of **LA,** denoted by **Base (LA)** and called the base address of LA.

Using this address Base (LA), the computer calculates the address of any element of LA by the following formula:

LOC (LA[K]) = Base(LA) + w(K-lower bound)

The above formula is known as indexing formula, where w is the no. of byte required by each element. K is the position on which one can locate and access the content of LA [K].

Operations on Arrays:

Various operations that can be performed on an array are

Traversing:

This is used visiting all elements in an array. A simplified algorithm is presented as below:

Algorithm : TRAVERSE_ARRAY ( )

Input : An array **LA** with elements.

Output : According to **PROCESS ( )**

Data Structure :Array LA[LB…..UB]

// LB and **UB** the lower and Upper Bound of array index.

Steps:

1. [initialization]

   i=LB // Start from first location L

2. [check the array up to the upper bound]

   while i<= UB do

3. [Traverse the array according to process]

   Process (LA [i])

4. [increment the counter]

   i=i+1 // Move to the next location

5. End While

6. Stop.

Note: Here PROCESS() is a procedure which called for an element when an action performs. For example, display the element on the screen.

Program: 1

```
class Traverse {
    public static void main(String args[]){
         // taking an array
        int a[] = { 5,8,3,6,2 };
        int i, x;
        // Iterating over an array
        for (i = 0; i < a.length; i++) {
            // accessing each element of array
            x = a[i];
            System.out.print(x + " ");
        }
    }
}
```

Out put:

5 8 3 6 2

Searching:

This is used to searching a particular element in an array. The algorithm presented below:

Algorithm : SEARCHING_ARRAY ( )

Input : An array **LA** with elements.

Output : According to **PROCESS ( )**

Data Structure :Array LA[LB…..UB]

// L B and **UB** the lower and Upper Bound of array index.

Steps:

1. **[Initialization]**

   Temp=LB // Counter Variable

2. **[Consider the first element of the array as largest]**

   Large=List[temp]

3. **[Find the smallest and largest number from array]**

   Repeat while Temp<=UB

   If LA[Temp]>Large

   Large=LA[Temp]

4. **[Increment the counter]**

   Temp=Temp+1

5. Out Put Largest Element
6. Exit

Program:2 To find the greatest number from an array.

```
#include<stdio.h>
#include<conio.h>
void main()
{
  int arr[5]={5,12,20,8,10};
  int  Temp=0;
  int Large=arr[0]
  clrscr();
  for(; Temp<=4; Temp++ )
            {
                  if(arr[Temp]>Large)
                       Large=arr[Temp];
             }
       }
   printf("%d", Large) ;
   getch();
}
```

- **Insertion:**

This is used to inserting a particular element in an array. The algorithm presented below:

Algorithm : INSERT (VALUE, POSITION)

Input : VALUE is the element, **POSITION** is the index of the element where it is to be inserted.

Output : Array enriched with VALUE.

Data Structure :Array LA[LB…..UB]

// LB and **UB** the lower and Upper Bound of Array

Steps:

1. **[Initialization]**

   i = len // i counter variable, len is
   // the length of array

2. **[Shift the element down by one position]**

   Repeat while i > =pos // pos is the index of the
   //element where element is to be
   //inserted

   LA[i+1]= LA[i]

3. **[decrement the counter]**

   i=i-1

4. **[Insert the element at required position]**

   LA[pos]= VALUE;

5. **[Reset Length of ARRAY]**

   len=len+1

6. Display the new list of arrays

7. Exit.

Program: 3 To insert an element in array at specified location

```
1.import java.util.Scanner;
2.public class Insert_Array
3.{
4.public static void main(String[] args)
5.{
6.int n, pos, x;
7.Scanner s = new Scanner(System.in);
8.System.out.print("Enter no. of elements you want in array:");
9.n = s.nextInt();
10.int a[] = new int[n+1];
11.System.out.println("Enter all the elements:");
12.for(int i = 0; i < n; i++)
13.{
14.a[i] = s.nextInt();
15.}
16.System.out.print("Enter the position where you want to insert
  element:");
17.pos = s.nextInt();
```

```
18.System.out.print("Enter the element you want to
insert:");
19.x = s.nextInt();
20.for(int i = (n-1); i >= (pos-1); i--)
21.{
22.a[i+1] = a[i];
23.}
24.a[pos-1] = x;
25.System.out.print("After inserting:");
26.for(int i = 0; i < n; i++)
27.{
28.System.out.print(a[i]+",");
29.}
30.System.out.print(a[n]);
31.}
32.}
```

Deletion :

This is used to delete a particular element in an array. The algorithm presented below:

Algorithm : DELETEA (a[ ], pos, n)

a[ ] : Linear Array

pos : position at which element to be deleted.

n : Total no of array

Steps:

1.[Initialization]

value= a[pos]

j=pos

2. [Delete the element]

Repeat while j= n-1

[shifting elements 1 position upwards]

a[j]=a[j+1]

[increment the value of j by 1]

j=j+1

end of loop

3. Reset n=n-1

4. Display the new list of element of arra

5. END

Program: To delete the element from array at desired position

```
1. import java.util.Scanner;
2.public class Delete
3.{
4.public static void main(String[] args)
5.{
6.int n, x, flag = 1, loc = 0;
7.Scanner s = new Scanner(System.in);
8.System.out.print("Enter no. of elements you want in
  array:");
9.n = s.nextInt();
10.int a[] = new int[n];
11.System.out.println("Enter all the elements:");
12.for (int i = 0; i < n; i++)
13.{
14.a[i] = s.nextInt();
15.}
16.System.out.print("Enter the element you want to
  delete:");
17.x = s.nextInt();
18.for (int i = 0; i < n; i++)
19.{
```

```
20.if(a[i] == x)
21.{
22.flag =1;
23.loc = i;
24.break;
25.}
26.else
27.{
28.flag = 0;
29.}
30.}
31.if(flag == 1)
32.{
33.for(int i = loc+1; i < n; i++)
34.{
35.a[i-1] = a[i];
36.}
37.System.out.print("After Deleting:");
38.for (int i = 0; i < n-2; i++)
39.{
40.System.out.print(a[i]+",");
41.}
42.System.out.print(a[n-2]);
43.}
44.else
45.{
46.System.out.println("Element not
found");
47.}
48.}
49.}
```

Merging Two Arrays:

This is used to merging two arrays in one. The algorithm presented below:

Algorithm : MERGE (aaray1[], array2[], array3[],p,q)

array1[] : First Array with some element

array2[] : Second Array with some element

array3[] : Third Array with some element

p : No of element in First Array

q : No of element in Second Array

Steps:

1. [Initialization]

i=j=k=LB // Lower Bound to 0

2.[Merging both arrays]

(i) [Assign the elements of array1 and array2 to array3 accordingly]

```
Repeat while i<p and j< q
    If(array1[i]<=array2[j])
            array3[k]= array[i]
            set  i++
    else
            array3[k]= array[j]
            set  j++
            set c++
```

(ii) [Assign remaining element of array1 to array 3]

Repeat while i<p

array3[k]=array1[i]

k++

(iii) [Assign remaining element of arra2 to array3]

Repeat while j<q

array3[k]=array2[j]

k++

3. Display the elements of array3

4. Exit

Multi Dimensional Array:

Most Programming languages allow two dimensional and three-dimensional arrays i.e. arrays where elements are referred by two or three sub-scripts.

Two Dimensional Array:

Two dimensional array (alternatively termed as matrix) are the collection of homogeneous elements where elements are ordered in a no. of rows and columns.

For Ex: A matrix A m×n, where m denote no of rows & n denotes no of columns.

Syntax:

data type array name[rows][columns]

i.e. int a[3][3]

in two dimensional array A, the elements of A form a rectangular array with m rows and n columns. Note that a row is a horizontal list of elements and a column is vertical list of elements the following figure shows the 2-dimensional array having 3-rows and 3-columns.

a00	a01	a02
a10	a11	a12
a20	a21	a22

Representation of 2-dimensional array in memory:

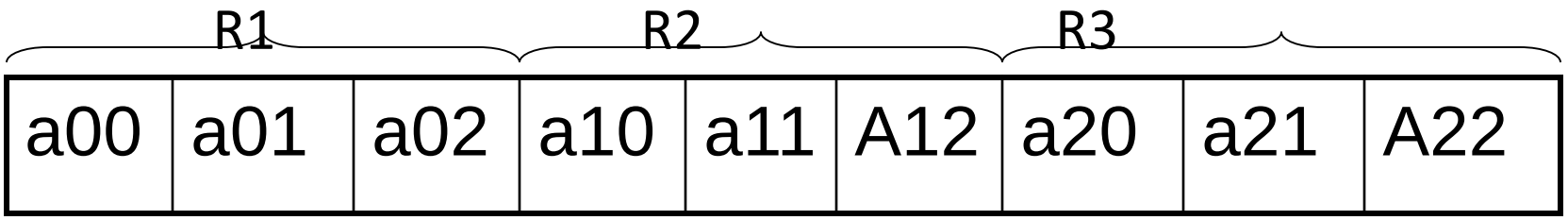
Like one dimensional array, matrix is also stored in contiguous memory location. There are two conventions of storing any matrix in memory.

1.Row Major Order

2.Column Major Order

1.Row Major Order:

In RMO, elements of the matrix are stored on row by row basis, i.e. all the elements of first row then second row & so on.

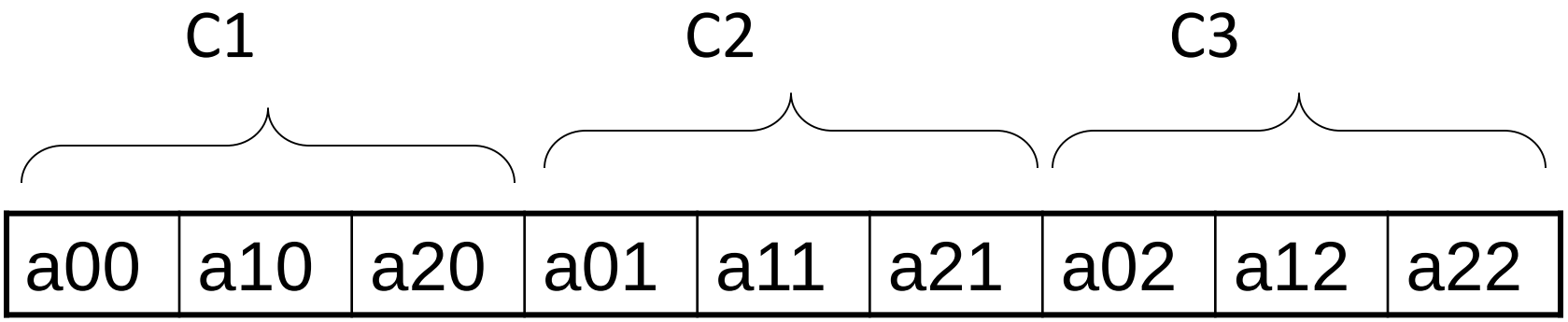


RMO(Row Major order)

2.column Major Order:

In the CMO, all the elements are stored column by column.

For e.g. consider a matrix shows above



CMO(Column Major order)

Address of a element:

Row Major Order:

Address of

$$A[I][J] = B + W * [N * (I - Lr) + (J - Lc)]$$

Column Major Order:

Address of

$$A[I][J] = B + W * [(I - Lr) + M * (J - Lc)]$$

Where,

- **B = Base address**
- **I = Row subscript of element whose address is to be found**
- **J = Column subscript of element whose address is to be found**
- **W = Storage Size of one element stored in the array (in byte)**
- **Lr = Lower limit of row/start row index of matrix, if not given assume 0 (zero)**
- **Lc = Lower limit of column/start column index of matrix, if not given assume 0 (zero)**
- **M = Number of row of the given matrix N = Number of column of the given matrix**

A 3 x 4 integer array A is as below:

Subscript	Elements	Address
10	(1,1)	1000
60	(1,2)	1002
30	(1,3)	1004
55	(1,4)	1006
20	(2,1)	1008
90	(2,2)	1010
80	(2,3)	1012
65	(2,4)	1014
50	(3,1)	1016
40	(3,2)	1018
75	(3,3)	1020
79	(3,4)	1022

A 3 x 4 integer array A is as below:

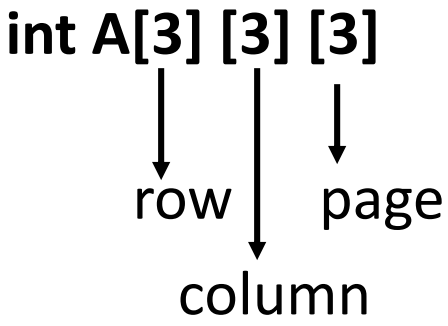
Elements	Subscript	Address
10	(1,1)	1000
20	(2,1)	1002
50	(3,1)	1004
60	(1,2)	1006
90	(2,2)	1008
40	(3,2)	1010
30	(1,3)	1012
80	(2,3)	1014
75	(3,3)	1016
55	(1,4)	1018
65	(2,4)	1020
79	(3,4)	1022

Three-Dimensional Array:

A 3-dimensional array can be compared with a book whereas 2-dimensional arrays and 1-dimensional array can be compare with a page and line respectively.

Here 3 major dimensions can be termed as ***row, column*** and ***page.***

For Ex:



The syntax for 3-d array is

data type array name [rows][columns][pages]

e.g. int A[3][3][3]

Where A is a array of integer type &

First index = No of rows (no of element in a column)

Second index = No of column (no of element in a row)

Third index = No of pages.

Storing a 3-D array means storing the pages one by one.

Storing a page is same as storing a 2-D array. Thus if the elements in a page stored in row major order we termed that 3-D array is also in row major order. The following formula is taken in to consideration of row major order of a 3-D array.

Row Major Order:

Add[Aijk]= Base address+ No. of element up to (k-1)th pages + no of elements on kth page up to (i-1) rows + no of element on kth page, in ith row up to (j-1)th column.

add[Aijk] =

$$\text{base}(A) + w * [N(k-1) + N(k)(i-1) + N(K)(i)(j-1)]$$

Column Major Order:

Add[Aijk]= Base address+ No. of element up to (k-1)th pages + no of elements on kth page up to (j-1)column + no of element on kth page, in jth column up to (i-1)th row.

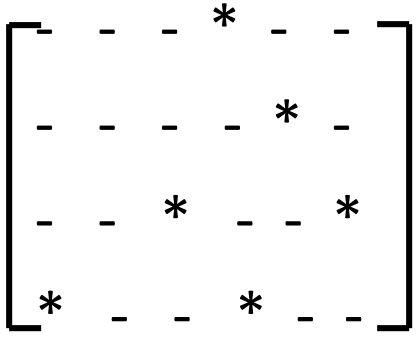
Add[Aijk]=

$$base(A) + w*[N(k-1)+ N(k)(j-1) + N(k)(j)(i-1)]$$

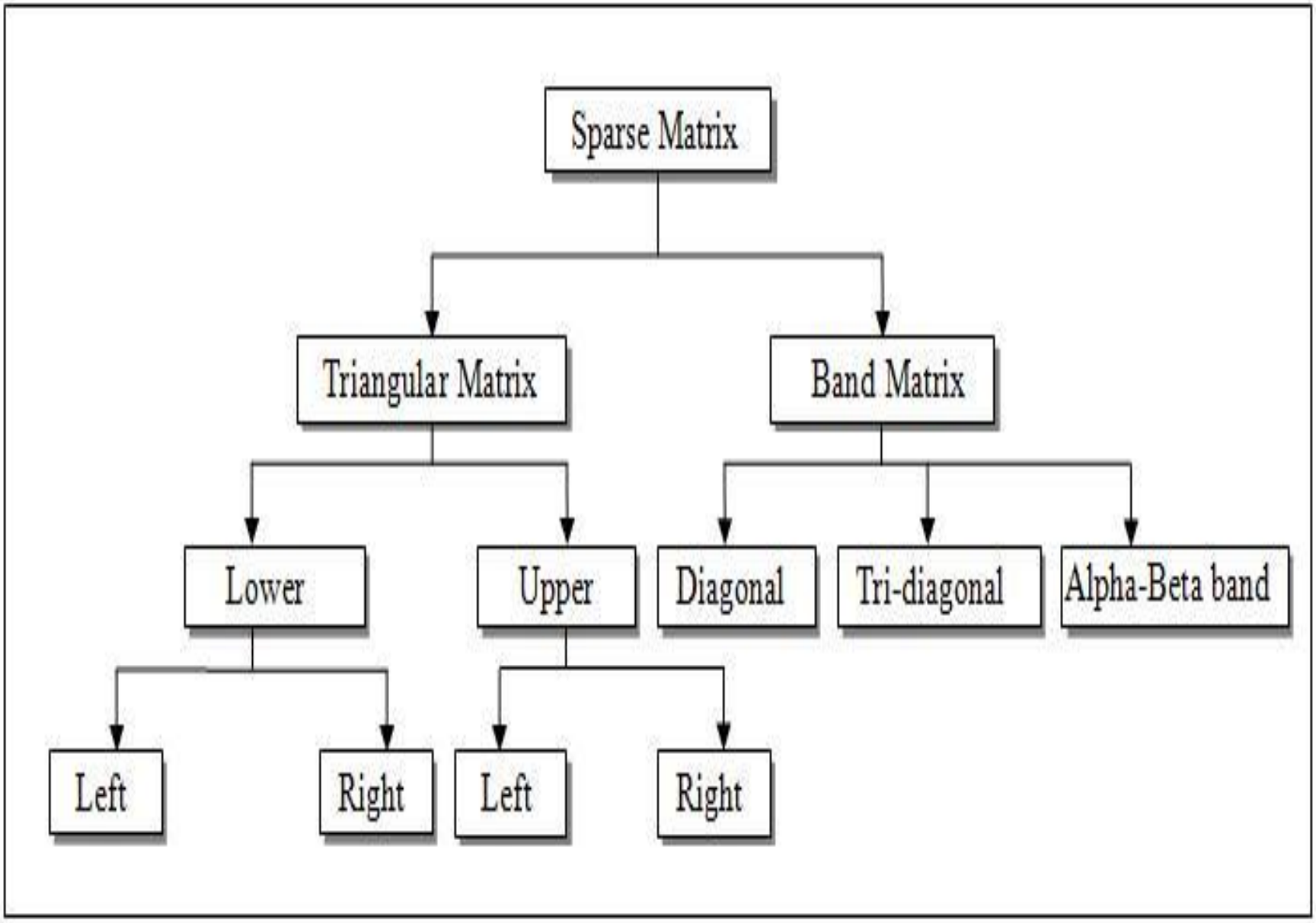
Sparse Matrix:

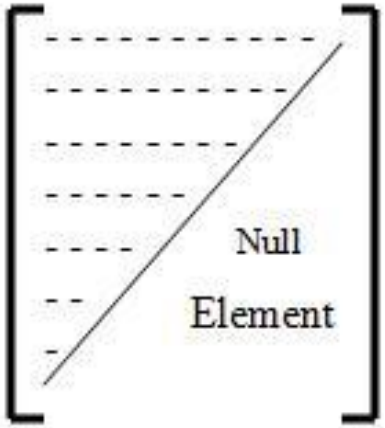
A sparse matrix is a two dimensional array having the value of majority element as null.

Following is the sparse matrix where * denotes the elements having non-null value.

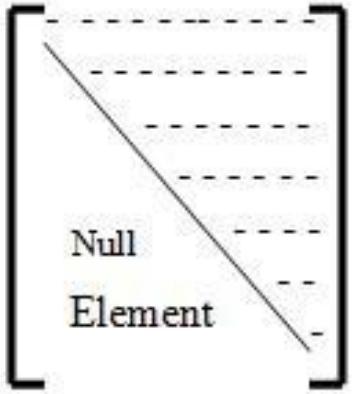


Some well known sparse matrices which are symmetric in form can be classified as follows.

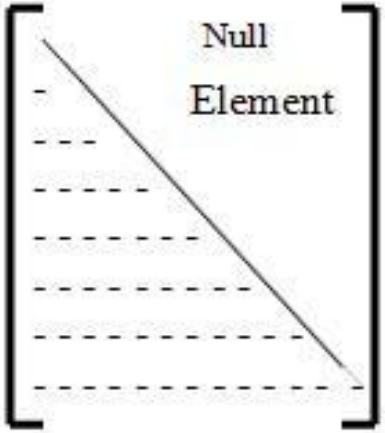

Sparse Matrix
Triangular Matrix
Band Matrix
Lower
Upper
Diagonal
Tri-diagonal
Alpha-Beta band
Left
Right
Left
Right



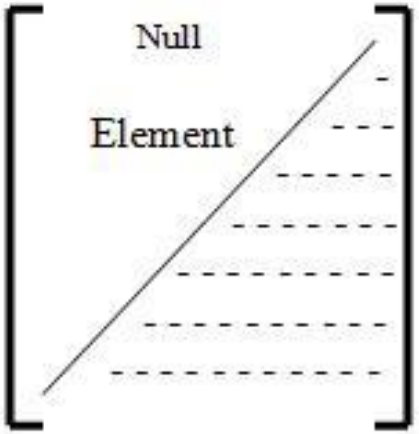
Upper Left Triangular



Upper Right Triangular

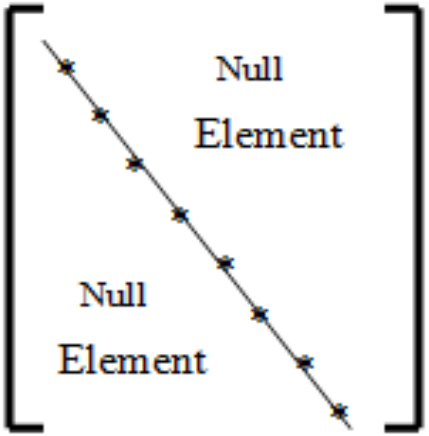


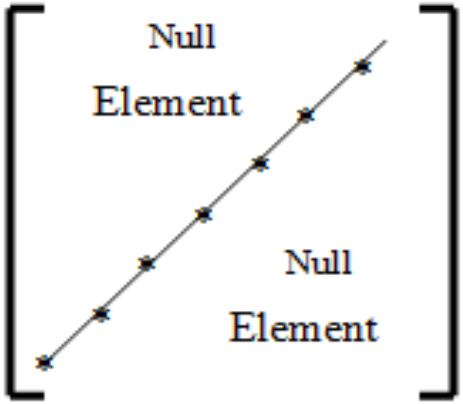
Lower Left Triangular



Lower Right Triangular

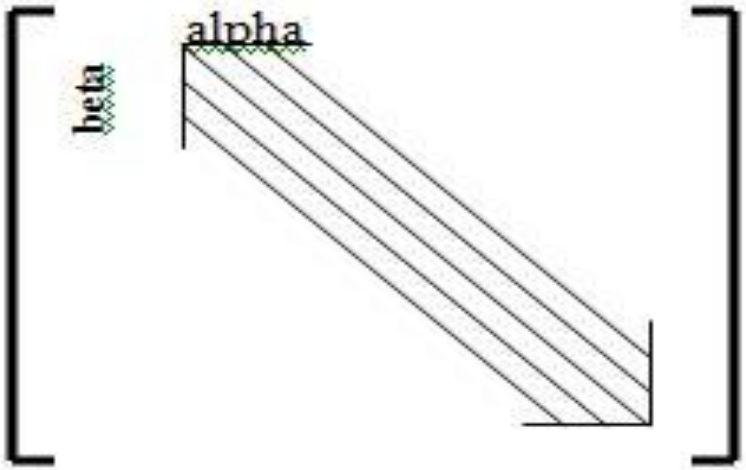
Various Triangular Matrices





Diagonal Matrices

Tridiagonal Matrices



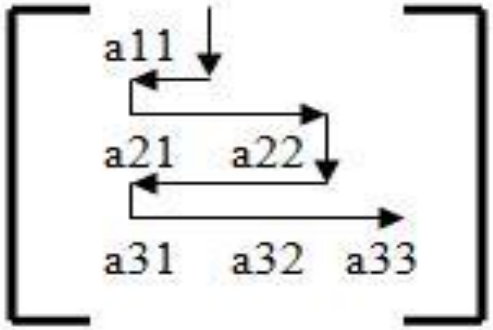
Alpha-Beta Band Matrix

Alpha-Beta Band Matrix: non-null elements are only on alpha-upper diagonal and beta-lower diagonal matrix

Memory representation of lower triangular Matrix:

There is a triangular array ‘A’ in memory. It would be wasteful to store those entries above the main diagonal of A, since they are all 0. hence we store only the other entries of matrix ‘A’ in a linear array B as indicated by arrow

Row Major Order:



so Array B is B[0]=a11, B[1]=a21, B[2]=a22 and .....so on

B[0]	B[1]	B[2]	B[3]	B[4]	B[5]
a11	a21	a22	a31	a32	a33

Observe that array B will contain only

1+2+3+……+n= (n+1)(n+2)/2 elements.

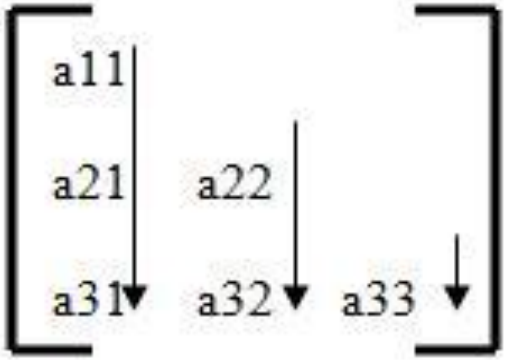
This is about as many elements as a 2-D n x n array.

Address of a element:

We can find the address of element as

Add(a[i][j])= Base(A)+ w[i(i-1)/2+(j-1)] where i >=0 and j<=n

Column Major order:



so Array B is B[0]=a11, B[1]=a21, B[2]=a31, B[3]=a22 and .....so on

B[0]	B[1]	B[2]	B[3]	B[4]	B[5]
a11	**a21**	**a31**	**a22**	**a32**	**A33**

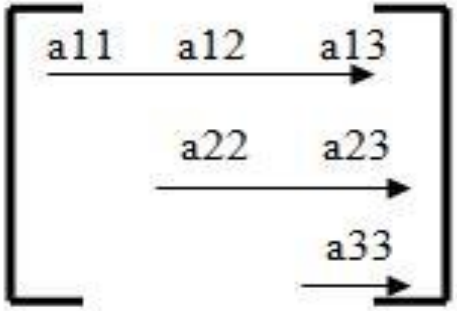
We can find the address of element as

Add(a[i][j])= Base(A) + w[(j-1)(n-j/2)+(i-1)] where i >=0 and

j<=n

Memory representation of upper triangular Matrix:

Row Major Order:



so Array B is B[0]=a11, B[1]=a12, B[2]=a13, B[3]=a22 and .....so on

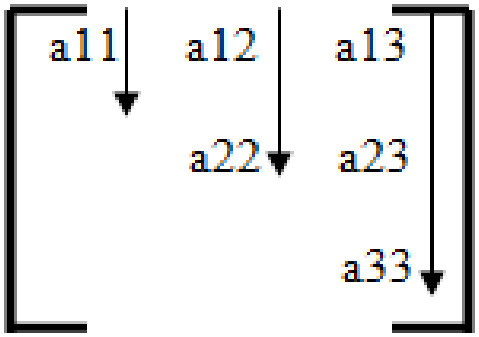
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]
a11	**a12**	**a13**	**a22**	**a23**	**a33**

Address of a element:

We can find the address of element as

Add(a[i][j][k])= Base(A)+ w[(i-1)*(n-i/2)+(j-1)]

Column Major Order:



so Array B is B[0]=a11 B[1]=a12, B[2]=a22, B[3]=a13 and .....so on

B[0]	B[1]	B[2]	B[3]	B[4]	B[5]
a11	**a12**	**a22**	**a13**	**a23**	**a33**

Address of a element:

We can find the address of element as

Add(a[i][j][k])= Base(A)+ w[j(j-1)/2 + i-1]

Array Advantages:

- Simple and easy to create
- Array will always hold similar kind of information.
- Continuous memory location
- Light on memory usage compared to other structures
- Can be used to create other useful data structures (queues, stacks).
- Arrays also are among the most compact data structures; storing 100 integers in an array takes only 100 times the space required to store an integer, plus perhaps a few bytes of overhead for the whole array.

Array Disdvantages:

- Rigid structure.
- cannot be dynamically resized in most languages.
- in case of static arrays the size of the array should be known up prior i.e before the compile time.
- Insertion/deletion in case of arrays is very costly since a lot of data movement has to be done.
- waste of memory.
- we cant change the size of array during run time.

Assignment No .1

1. What is data structure?

2. List out the areas in which data structures are applied extensively?

3. What are the major data structures used in the following areas : RDBMS, Network data model and Hierarchical data model.

4. What do you mean by: Syntax Error, Logical Error, Run time Error?

5. What's the major difference in between Storage structure and file structure and how?

Assigment

1. Give a brief description of a) traversing b) sorting c) searching

2. What are the limitations of arrays?

3. Explain worst case and average case of an algorithm.

4. What do you understand by Subalgorithms?

5. Write an algorithm to multiply mxn matrices?